```python
In [2]: import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import seaborn as sns
```

```python
In [3]: import warnings
        warnings.filterwarnings("ignore")
        sns.set(style="whitegrid")
```

```python
In [4]: df = pd.read_csv("AIML Dataset.csv")
```

```python
In [5]: df.head()
```

Out[5]:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1	0
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1	0
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0	0

```python
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column          Dtype
---  ------          -----
 0   step            int64
 1   type            object
 2   amount          float64
 3   nameOrig        object
 4   oldbalanceOrg   float64
 5   newbalanceOrig  float64
 6   nameDest        object
 7   oldbalanceDest  float64
 8   newbalanceDest  float64
 9   isFraud         int64
 10  isFlaggedFraud  int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
```

```python
In [8]: df.columns
```

```
Out[8]: Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
               'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
               'isFlaggedFraud'],
              dtype='object')
```

```python
In [9]: df["isFraud"].value_counts()
```

```
Out[9]: isFraud
        0    6354407
        1       8213
        Name: count, dtype: int64
```

```python
In [10]: df["isFraud"].value_counts()
```

```
Out[10]: isFraud
         0    6354407
         1       8213
         Name: count, dtype: int64
```

```python
In [11]: df["isFlaggedFraud"].value_counts()
```

```
Out[11]: isFlaggedFraud
         0    6362604
         1         16
         Name: count, dtype: int64
```

```python
In [12]: df.isnull().sum()
```

```
Out[12]: step              0
         type              0
         amount            0
         nameOrig          0
         oldbalanceOrg     0
         newbalanceOrig    0
         nameDest          0
         oldbalanceDest    0
         newbalanceDest    0
         isFraud           0
         isFlaggedFraud    0
         dtype: int64
```

```python
In [13]: df.shape[0]
```

```
Out[13]: 6362620
```

```python
In [14]: round((df["isFraud"].value_counts()[1]/ df.shape[0])*100,2)
```

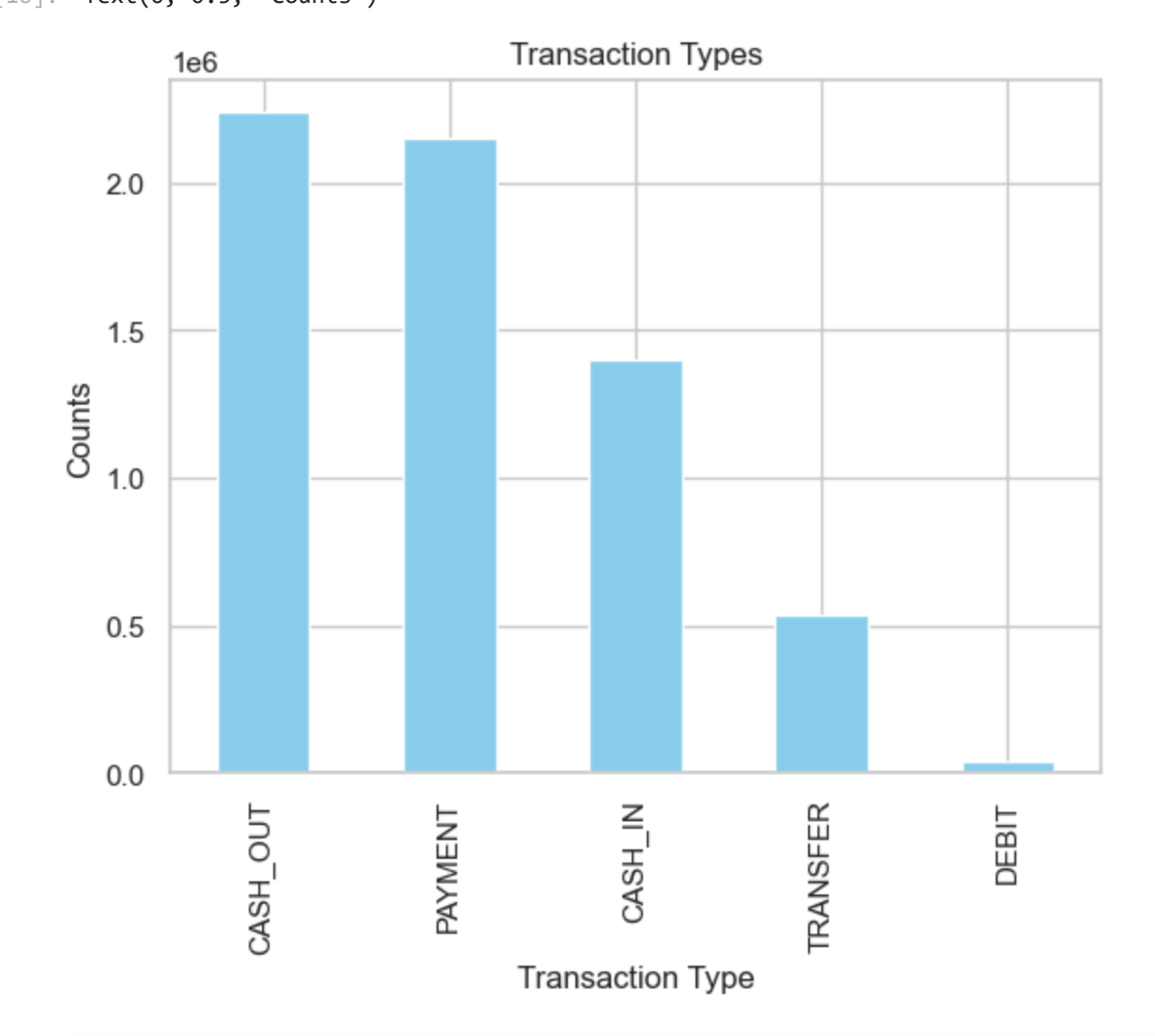
```
Out[14]: np.float64(0.13)
```

```python
In [15]: df.columns
```

```
Out[15]: Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
                'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
                'isFlaggedFraud'],
               dtype='object')
```

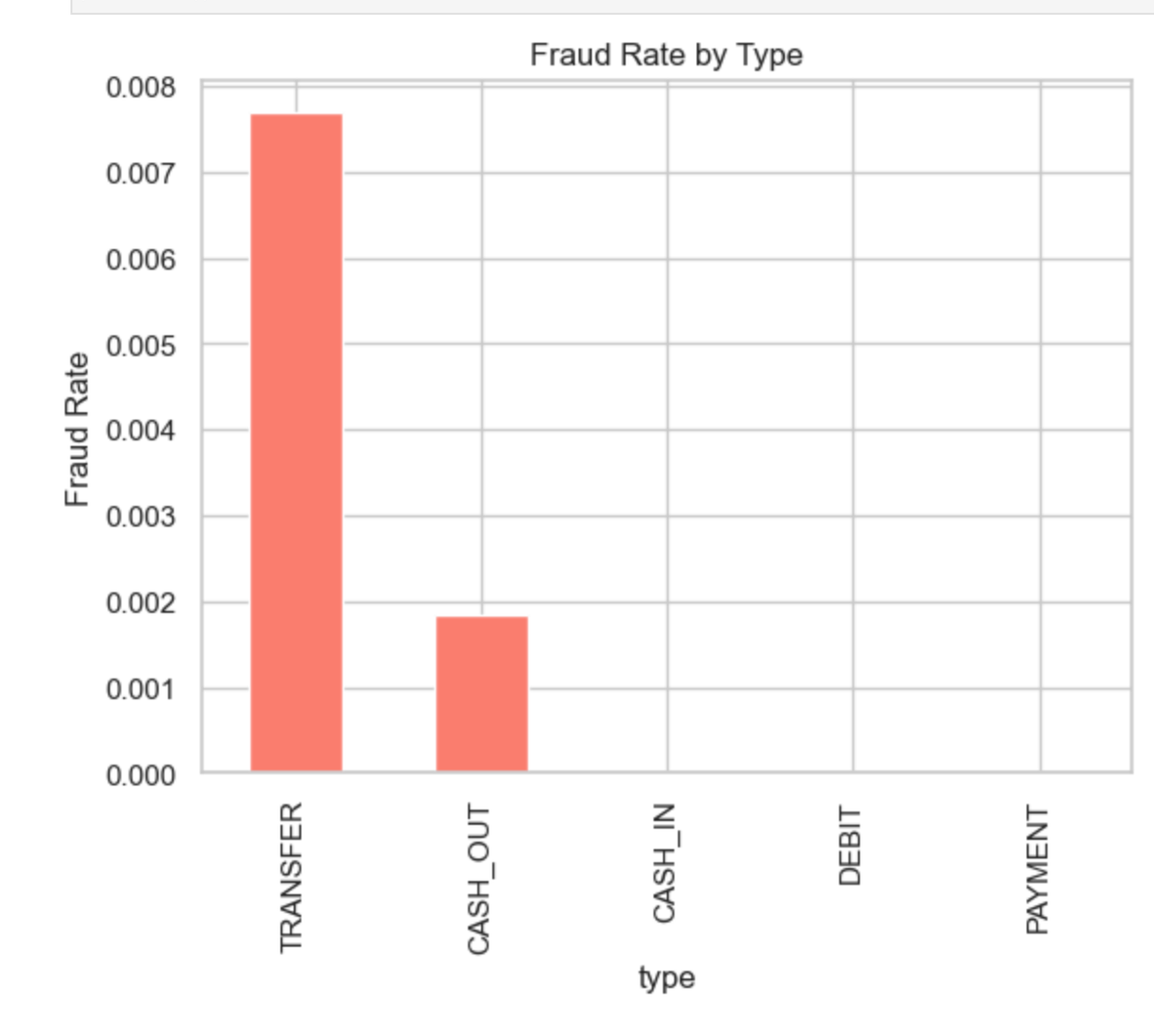
```python
In [16]: df["type"].value_counts().plot(kind="bar", title="Transaction Types", color="skyblue")
         plt.xlabel("Transaction Type")
         plt.ylabel("Counts")
```

```
Out[16]: Text(0, 0.5, 'Counts')
```




```python
In [17]: fraud_by_type = df.groupby("type")["isFraud"].mean().sort_values(ascending=False)
```

```python
In [18]: fraud_by_type.plot(kind = "bar", title = "Fraud Rate by Type", color="salmon")
         plt.ylabel("Fraud Rate")
         plt.show()
```



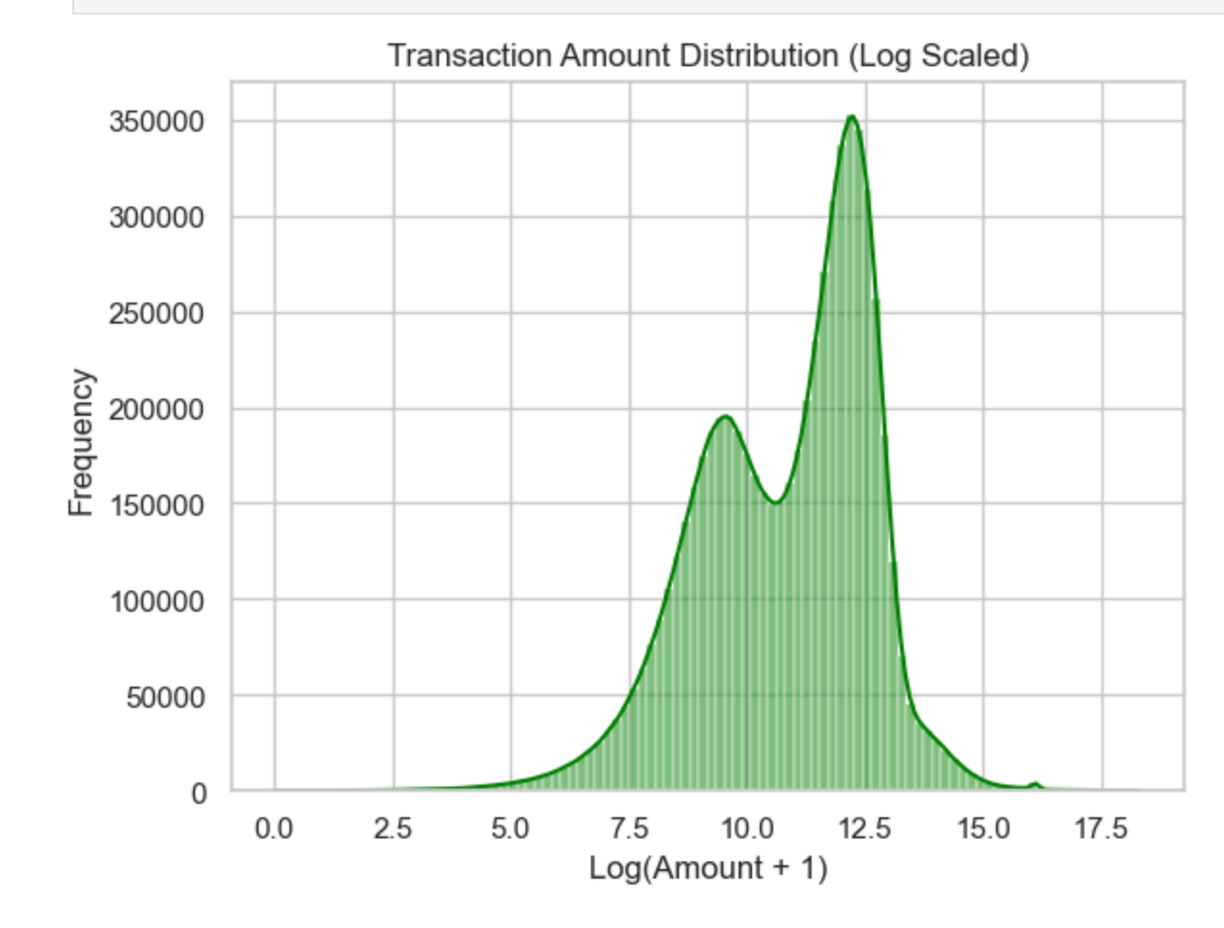

```python
In [19]: fraud_by_type
```

```
Out[19]: type
         TRANSFER    0.007688
         CASH_OUT    0.001840
         CASH_IN     0.000000
         DEBIT       0.000000
         PAYMENT     0.000000
         Name: isFraud, dtype: float64
```

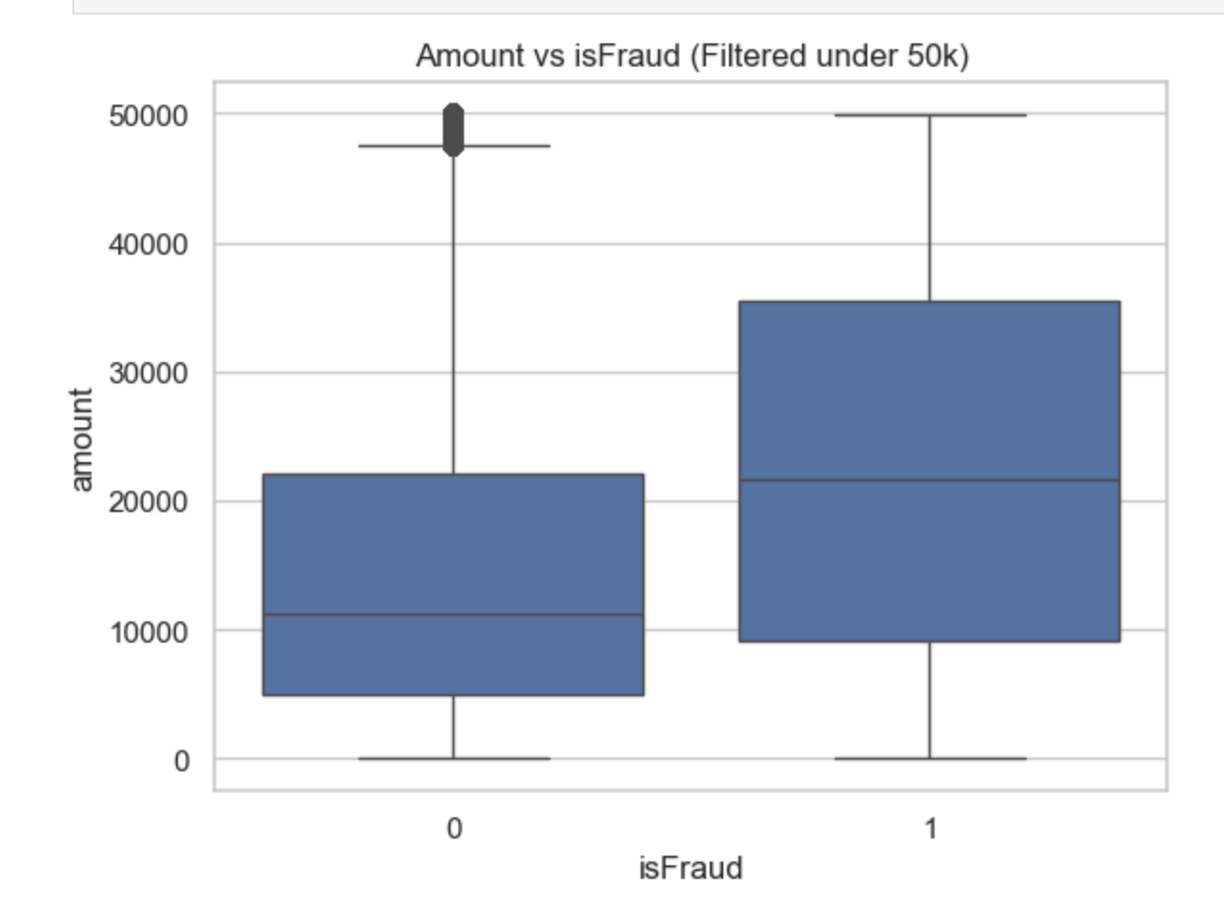
```python
In [20]: df["amount"].describe().astype(int)
```

```
Out[20]: count     6362620
         mean       179861
         std        603858
         min             0
         25%         13389
         50%         74871
         75%        208721
         max      92445516
         Name: amount, dtype: int64
```

```python
In [24]: sns.histplot(np.log1p(df["amount"]), bins=100, kde=True, color="green")
         plt.title("Transaction Amount Distribution (Log Scaled)")
         plt.xlabel("Log(Amount + 1)")
         plt.ylabel("Frequency")
         plt.show()
```




```python
In [25]: sns.boxplot(data = df[df["amount"] <50000], x = "isFraud", y = "amount")
         plt.title("Amount vs isFraud (Filtered under 50k)")
         plt.show()
```




```python
In [26]: df.columns
```

```
Out[26]: Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
                'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
                'isFlaggedFraud'],
               dtype='object')
```

```python
In [27]: df["balanceDiffOring"] = df["oldbalanceOrg"] - df["newbalanceOrig"]
         df["balanceDiffDest"] = df["newbalanceDest"] - df["oldbalanceDest"]
```

```python
In [30]: (df["balanceDiffOring"] <0).sum()
```

```
Out[30]: np.int64(1399253)
```

```python
In [31]: (df["balanceDiffDest"] <0).sum()
```

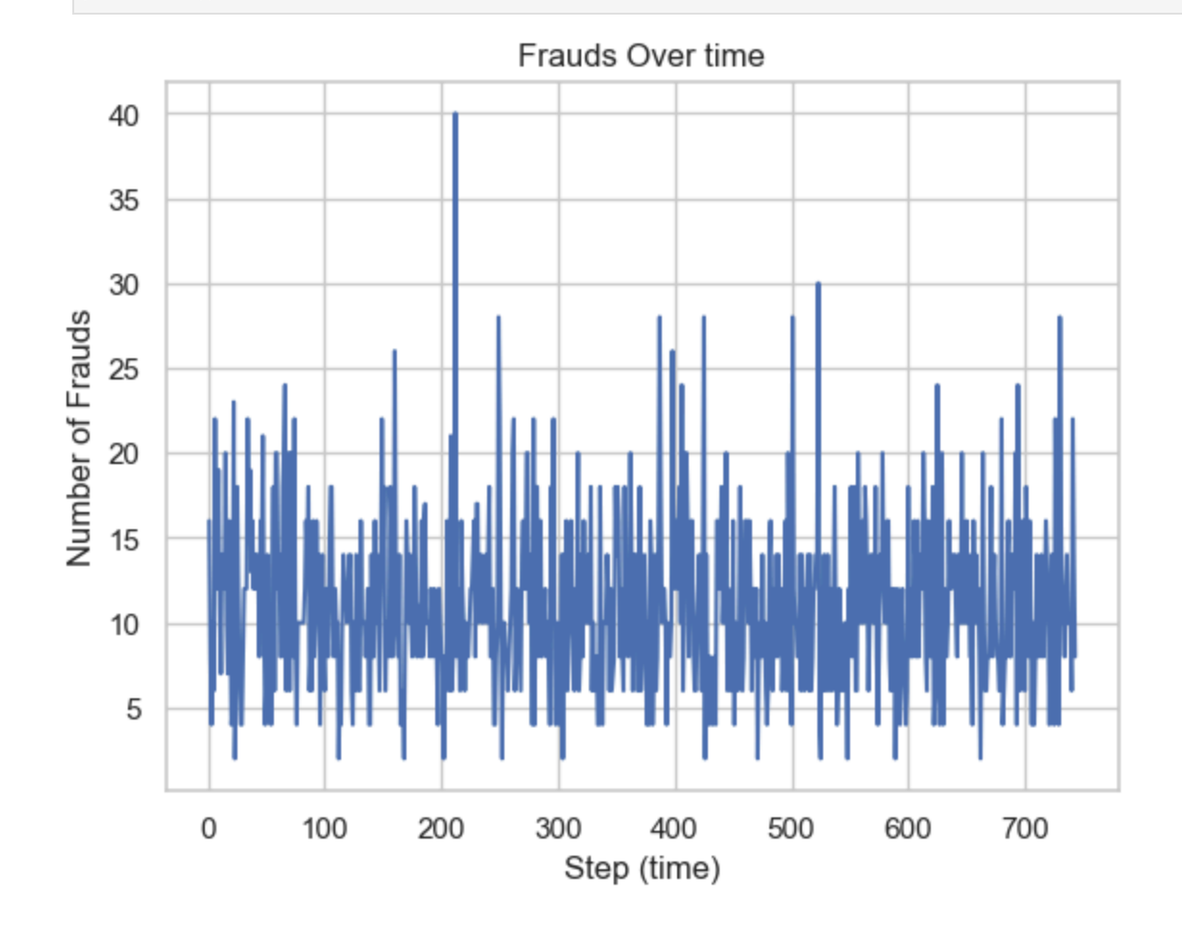
```
Out[31]: np.int64(1238864)
```

```python
In [32]: df.head(2)
```

Out[32]:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOring	balanceDiffDest
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0	9839.64	0.0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0	1864.28	0.0

```python
In [36]: frauds_per_step = df[df["isFraud"] == 1]["step"].value_counts().sort_index()
         plt.plot(frauds_per_step.index, frauds_per_step.values, label="Frauds per Step")
         plt.xlabel("Step (time)")
         plt.ylabel("Number of Frauds")
         plt.title("Frauds Over time")
         plt.grid(True)
         plt.show()
```




```python
In [37]: df.drop(columns="step", inplace=True)
```

```python
In [38]: df.head(4)
```

Out[38]:

	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOring	balanceDiffDest
0	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0	9839.64	0.0
1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0	1864.28	0.0
2	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1	0	181.00	0.0
3	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1	0	181.00	-21182.0

```python
In [40]: top_sender = df["nameOrig"].value_counts().head(10)
```

```python
In [41]: top_sender
```

```
Out[41]: nameOrig
         C1677795071    3
         C1999539787    3
         C724452879     3
         C1976208114    3
         C400299098     3
         C1784010646    3
         C1530544995    3
         C1065307291    3
         C545315117     3
         C1902386530    3
         Name: count, dtype: int64
```

```python
In [42]: top_receivers = df["nameDest"].value_counts().head(10)
```

```python
In [43]: top_receivers
```

```
Out[43]: nameDest
         C1286084959    113
         C985934102     109
         C665576141     105
         C2083562754    102
         C248609774     101
         C1590550415    101
         C1789550256     99
         C451111351      99
         C1360767589     98
         C1023714065     97
         Name: count, dtype: int64
```

```python
In [44]: fraud_users = df[df["isFraud"] == 1]["nameOrig"].value_counts().head(10)
```

```python
In [45]: fraud_users
```

```
Out[45]: nameOrig
         C1305486145    1
         C840083671     1
         C1420196421    1
         C2101527076    1
         C137533655     1
         C1118430673    1
         C749981943     1
         C1334405552    1
         C467632528     1
         C1364127192    1
         Name: count, dtype: int64
```

```python
In [46]: fraud_type = df[df["type"].isin(["TRANSFER", "CASH_OUT"])]
```

```python
In [47]: fraud_type.head(10)
```

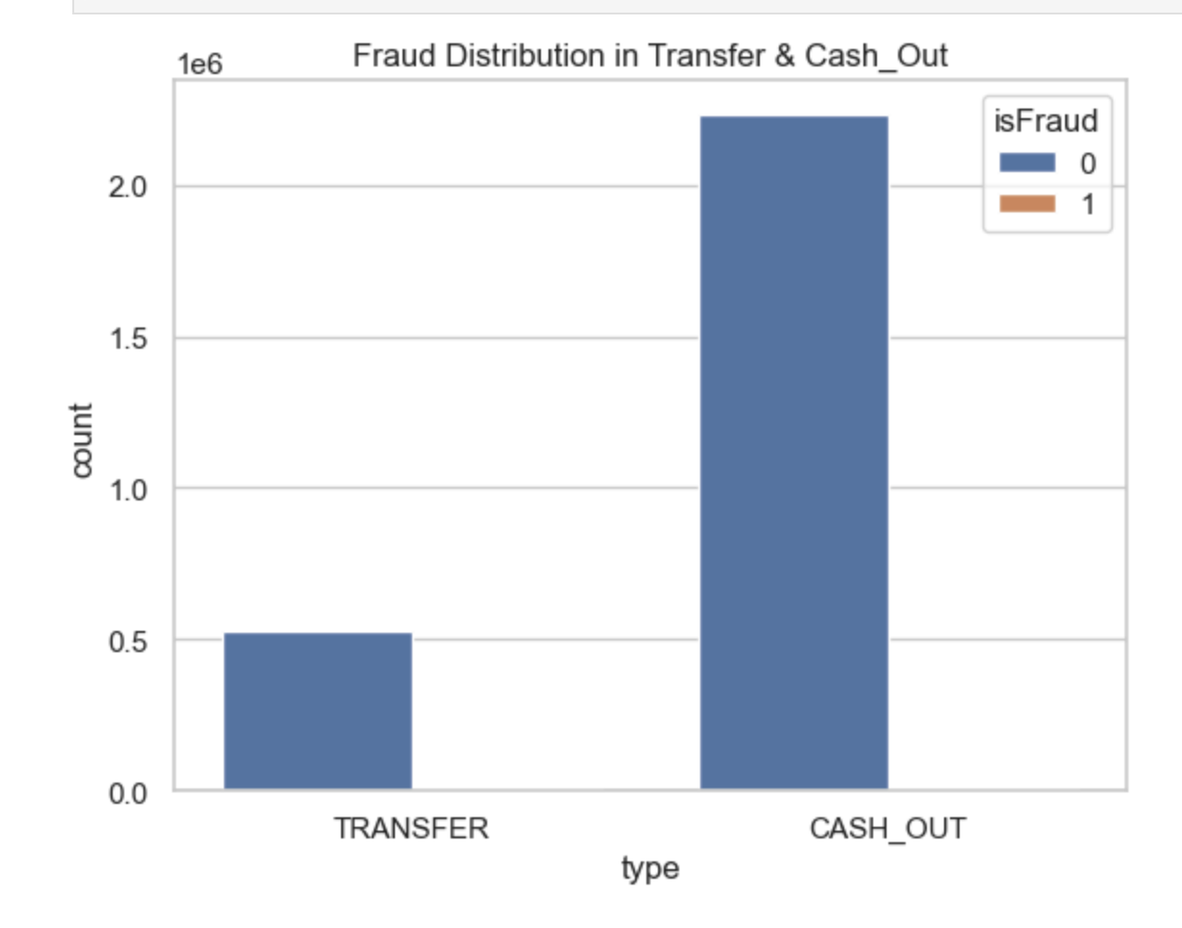
Out[47]:

	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOring	balanceDiffDest
2	TRANSFER	181.00	C1305486145	181.00	0.0	C553264065	0.0	0.00	1	0	181.00	0.00
3	CASH_OUT	181.00	C840083671	181.00	0.0	C38997010	21182.0	0.00	1	0	181.00	-21182.00
15	CASH_OUT	229133.94	C905080434	15325.00	0.0	C476402209	5083.0	51513.44	0	0	15325.00	46430.44
19	TRANSFER	215310.30	C1670993182	705.00	0.0	C1100439041	22425.0	0.00	0	0	705.00	-22425.00
24	TRANSFER	311685.89	C1984094095	10835.00	0.0	C932583850	6267.0	2719172.89	0	0	10835.00	2712905.89
42	CASH_OUT	110414.71	C768216420	26845.41	0.0	C1509514333	288800.0	2415.16	0	0	26845.41	-286384.84
47	CASH_OUT	56953.90	C1570470538	1942.02	0.0	C824009085	70253.0	64106.18	0	0	1942.02	-6146.82
48	CASH_OUT	5346.89	C512549200	0.00	0.0	C248609774	652637.0	6453430.91	0	0	0.00	5800793.91
51	CASH_OUT	23261.30	C2072313080	20411.53	0.0	C2001112025	25742.0	0.00	0	0	20411.53	-25742.00
58	TRANSFER	62610.80	C1976401987	79114.00	16503.2	C1937962514	517.0	8383.29	0	0	62610.80	7866.29

```python
In [48]: fraud_type["type"].value_counts()
```

```
Out[48]: type
         CASH_OUT    2237500
         TRANSFER     532909
         Name: count, dtype: int64
```

```python
In [49]: sns.countplot(data = fraud_type, x ="type", hue="isFraud" )
         plt.title("Fraud Distribution in Transfer & Cash_Out")
         plt.show()
```




```python
In [50]: df.columns
```

```
Out[50]: Index(['type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
                'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
                'isFlaggedFraud', 'balanceDiffOring', 'balanceDiffDest'],
               dtype='object')
```

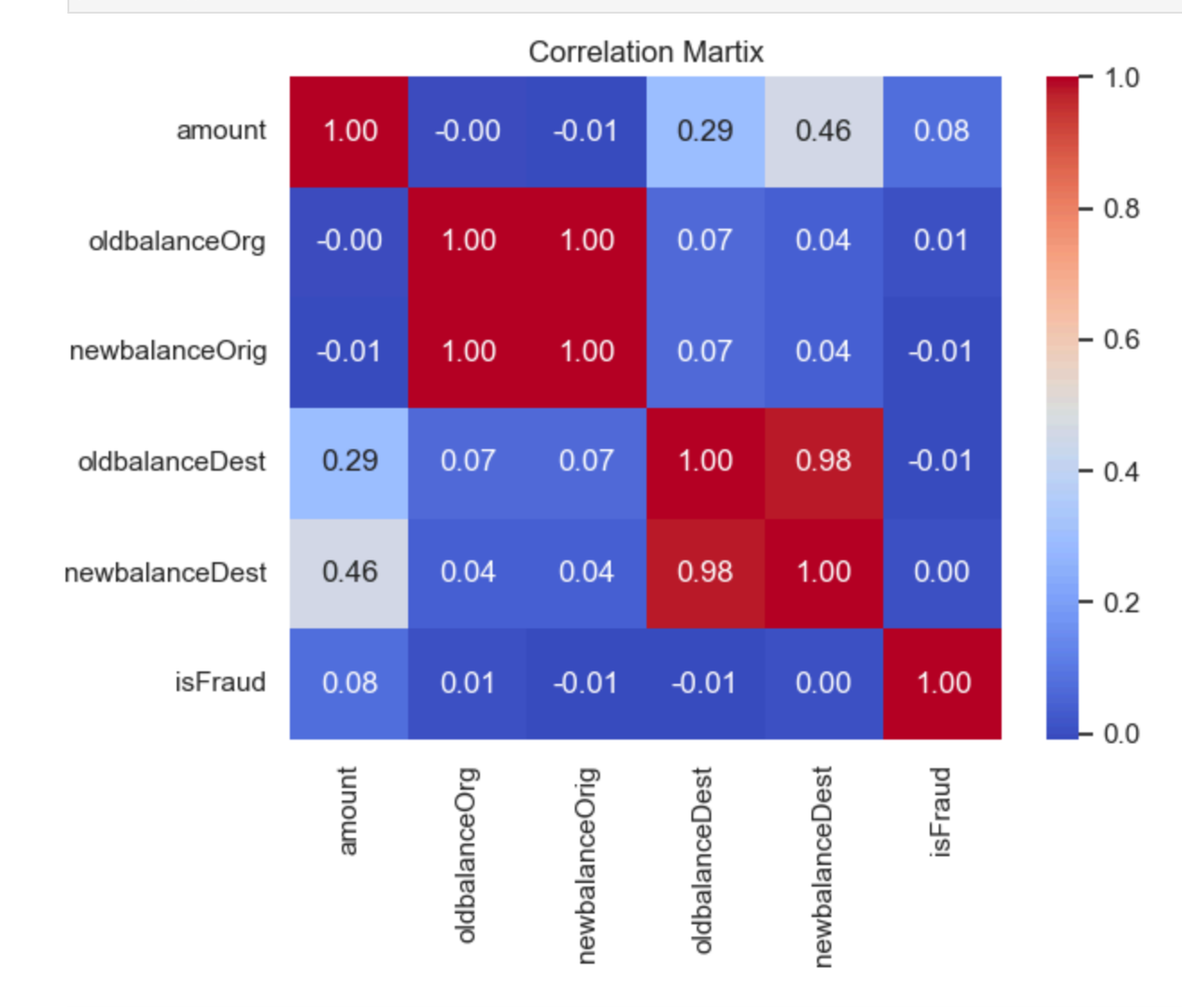
```python
In [53]: corr = df[["amount", "oldbalanceOrg", "newbalanceOrig",
                   "oldbalanceDest", "newbalanceDest", "isFraud"]].corr()
```

```python
In [54]: corr
```

Out[54]:

	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud
amount	1.000000	-0.002762	-0.007861	0.294137	0.459304	0.076688
oldbalanceOrg	-0.002762	1.000000	0.998803	0.066243	0.042029	0.010154
newbalanceOrig	-0.007861	0.998803	1.000000	0.067812	0.041837	-0.008148
oldbalanceDest	0.294137	0.066243	0.067812	1.000000	0.976569	-0.005885
newbalanceDest	0.459304	0.042029	0.041837	0.976569	1.000000	0.000535
isFraud	0.076688	0.010154	-0.008148	-0.005885	0.000535	1.000000

```python
In [55]: sns.heatmap(corr, annot=True, cmap="coolwarm", fmt = ".2f")
         plt.title("Correlation Martix")
         plt.show()
```




```python
In [56]: zero_after_transfer = df[
             (df["oldbalanceOrg"] >0)&
             (df["newbalanceOrig"] == 0) &
             (df["type"].isin(["TRANSFER", "CASH_OUT"]))
         ]
```

```python
In [57]: len(zero_after_transfer)
```

```
Out[57]: 1188074
```

```python
In [58]: zero_after_transfer.head()
```

Out[58]:

	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOring	balanceDiffDest
2	TRANSFER	181.00	C1305486145	181.0	0.0	C553264065	0.0	0.00	1	0	181.0	0.00
3	CASH_OUT	181.00	C840083671	181.0	0.0	C38997010	21182.0	0.00	1	0	181.0	-21182.00
15	CASH_OUT	229133.94	C905080434	15325.0	0.0	C476402209	5083.0	51513.44	0	0	15325.0	46430.44
19	TRANSFER	215310.30	C1670993182	705.0	0.0	C1100439041	22425.0	0.00	0	0	705.0	-22425.00
24	TRANSFER	311685.89	C1984094095	10835.0	0.0	C932583850	6267.0	2719172.89	0	0	10835.0	2712905.89

```python
In [59]: df["isFraud"].value_counts()
```

```
Out[59]: isFraud
         0    6354407
         1       8213
         Name: count, dtype: int64
```

```python
In [60]: from sklearn.model_selection import train_test_split
         from sklearn.preprocessing import StandardScaler
         from sklearn.linear_model import LogisticRegression
         from sklearn.metrics import classification_report, confusion_matrix
         from sklearn.pipeline import Pipeline
         from sklearn.compose import ColumnTransformer
         from sklearn.preprocessing import OneHotEncoder
```

```python
In [61]: df.head()
```

Out[61]:

	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOring	balanceDiffDest
0	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0	9839.64	0.0
1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0	1864.28	0.0
2	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1	0	181.00	0.0
3	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1	0	181.00	-21182.0
4	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0	0	11668.14	0.0

```python
In [64]: df_model = df.drop(["nameOrig", "nameDest", "isFlaggedFraud"], axis=1)
```

```python
In [65]: df_model.head()
```

Out[65]:

	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	balanceDiffOring	balanceDiffDest
0	PAYMENT	9839.64	170136.0	160296.36	0.0	0.0	0	9839.64	0.0
1	PAYMENT	1864.28	21249.0	19384.72	0.0	0.0	0	1864.28	0.0
2	TRANSFER	181.00	181.0	0.00	0.0	0.0	1	181.00	0.0
3	CASH_OUT	181.00	181.0	0.00	21182.0	0.0	1	181.00	-21182.0
4	PAYMENT	11668.14	41554.0	29885.86	0.0	0.0	0	11668.14	0.0

```python
In [67]: categorical = ["type"]
         numeric = ["amount", "oldbalanceOrg", "newbalanceOrig", "oldbalanceDest", "newbalanceDest" ]
```

```python
In [69]: y = df_model["isFraud"]
         X = df_model.drop("isFraud", axis=1)
```

```python
In [70]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y)
```

```python
In [71]: preprocessor = ColumnTransformer(
             transformers= [
                 ("num", StandardScaler(), numeric),
                 ("cat", OneHotEncoder(drop="first"), categorical)
             ],
             remainder="drop"
         )
```

```python
In [72]: pipeline = Pipeline([
             ("prep", preprocessor),
             ("clf", LogisticRegression(class_weight="balanced", max_iter=1000))
         ])
```

```python
In [73]: pipeline.fit(X_train, y_train)
```

Out[73]:

```
▸ Pipeline
  ▸ prep: ColumnTransformer
    ▸ num                 ▸ cat
    ▸ StandardScaler      ▸ OneHotEncoder
  ▸ LogisticRegression
```

```python
In [75]: y_pred = pipeline.predict(X_test)
```

```python
In [77]: print (classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.95      0.97   1906322
           1       0.02      0.94      0.04      2464

    accuracy                           0.95   1908786
   macro avg       0.51      0.94      0.51   1908786
weighted avg       1.00      0.95      0.97   1908786
```

```python
In [78]: confusion_matrix(y_test,y_pred)
```

```
Out[78]: array([[1803187,  103135],
                [    150,    2314]])
```

```python
In [80]: pipeline.score(X_test, y_test)*100
```

```
Out[80]: 94.58896911440047
```

```python
In [81]: import joblib
```

```python
In [82]: joblib.dump(pipeline, "fraud_dectection_pipline.pkl")
```

```
Out[82]: ['fraud_dectection_pipline.pkl']
```

```python
In [83]: import os
         os.getcwd()
```

Out[83]:
```
'C:\\Users\\sahil\\Fraud detection'
```

In [ ]: